

LAB 5: Supervised Learning

OBJECTIVE: Train models using supervised learning with labeled data.

DATASET: Wine Dataset

THEORY:

- Supervised learning trains a model on input-output pairs (features and labels).
- The model learns to map inputs to outputs by minimizing a loss function (e.g., cross-entropy for classification).
- Once trained, the model predicts labels for new/unseen data, and performance is evaluated using metrics like accuracy, confusion matrix, precision, recall, and F1-score.
- Tools: Scikit-learn and Keras simplify model building, training, and evaluation.

ALGORITHM:

- Load the dataset (Wine dataset)
- Split data into features (X) and target labels (y)
- Normalize features if necessary
- Split dataset into training and testing sets
- Train a classifier (e.g., Logistic Regression, MLP)
- Predict labels on the test set
- Evaluate performance using accuracy, classification report, and confusion matrix
- Optionally, visualize results

SOURCE CODE:

CELL 1: IMPORT LIBRARIES

```
In [13]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.utils import to_categorical
```

CELL 2: LOAD AND PREPROCESS DATA

```
In [14]: # Load Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert Labels to one-hot encoding
y_cat = to_categorical(y)

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_cat, test_size=0.2, random_state=42)
```

CELL 3: BUILD MLP MODEL

```
In [15]: model = Sequential()
model.add(Input(shape=(X.shape[1],))) # input layer
model.add(Dense(32, activation='relu')) # hidden layer
model.add(Dense(16, activation='relu')) # hidden layer
model.add(Dense(3, activation='softmax')) # output layer for 3 classes

model.compile(
    optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 32)	448
dense_7 (Dense)	(None, 16)	528
dense_8 (Dense)	(None, 3)	51

Total params: 1,027 (4.01 KB)

Trainable params: 1,027 (4.01 KB)

Non-trainable params: 0 (0.00 B)

CELL 4: TRAIN MODEL

```
In [16]: history = model.fit(X_train, y_train, epochs=50, batch_size=16, validation_split=0.2)
```

Epoch 1/50
8/8  1s 30ms/step - accuracy: 0.4397 - loss: 1.0854 - val_accuracy: 0.3793 - val_loss: 1.0630
Epoch 2/50
8/8  0s 10ms/step - accuracy: 0.6857 - loss: 0.9442 - val_accuracy: 0.7241 - val_loss: 0.9125
Epoch 3/50
8/8  0s 10ms/step - accuracy: 0.7798 - loss: 0.8382 - val_accuracy: 0.8276 - val_loss: 0.7975
Epoch 4/50
8/8  0s 10ms/step - accuracy: 0.8378 - loss: 0.7545 - val_accuracy: 0.8276 - val_loss: 0.7072
Epoch 5/50
8/8  0s 10ms/step - accuracy: 0.9082 - loss: 0.6371 - val_accuracy: 0.8621 - val_loss: 0.6296
Epoch 6/50
8/8  0s 10ms/step - accuracy: 0.9187 - loss: 0.5772 - val_accuracy: 0.8276 - val_loss: 0.5622
Epoch 7/50
8/8  0s 10ms/step - accuracy: 0.9031 - loss: 0.5047 - val_accuracy: 0.8621 - val_loss: 0.5038
Epoch 8/50
8/8  0s 10ms/step - accuracy: 0.9509 - loss: 0.4438 - val_accuracy: 0.8966 - val_loss: 0.4561
Epoch 9/50
8/8  0s 10ms/step - accuracy: 0.9365 - loss: 0.3872 - val_accuracy: 0.8966 - val_loss: 0.4143
Epoch 10/50
8/8  0s 14ms/step - accuracy: 0.9441 - loss: 0.3861 - val_accuracy: 0.8966 - val_loss: 0.3762
Epoch 11/50
8/8  0s 10ms/step - accuracy: 0.9527 - loss: 0.3227 - val_accuracy: 0.8966 - val_loss: 0.3406
Epoch 12/50
8/8  0s 9ms/step - accuracy: 0.9559 - loss: 0.2895 - val_accuracy: 0.8966 - val_loss: 0.3112
Epoch 13/50
8/8  0s 10ms/step - accuracy: 0.9375 - loss: 0.2625 - val_accuracy: 0.8966 - val_loss: 0.2891
Epoch 14/50
8/8  0s 10ms/step - accuracy: 0.9718 - loss: 0.2283 - val_accuracy: 0.8966 - val_loss: 0.2676
Epoch 15/50
8/8  0s 10ms/step - accuracy: 0.9582 - loss: 0.2264 - val_accuracy: 0.9310 - val_loss: 0.2479
Epoch 16/50
8/8  0s 10ms/step - accuracy: 0.9349 - loss: 0.2202 - val_accuracy: 0.9310 - val_loss: 0.2296
Epoch 17/50
8/8  0s 10ms/step - accuracy: 0.9724 - loss: 0.1826 - val_accuracy: 0.9310 - val_loss: 0.2130
Epoch 18/50
8/8  0s 10ms/step - accuracy: 0.9703 - loss: 0.1675 - val_accuracy: 0.9310 - val_loss: 0.1981
Epoch 19/50
8/8  0s 11ms/step - accuracy: 0.9543 - loss: 0.1806 - val_accuracy: 0.9310 - val_loss: 0.1847
Epoch 20/50
8/8  0s 10ms/step - accuracy: 0.9669 - loss: 0.1276 - val_accuracy: 0.9655 - val_loss: 0.1731

Epoch 21/50
8/8  0s 10ms/step - accuracy: 0.9473 - loss: 0.1507 - val_accuracy: 0.9655 - val_loss: 0.1627
Epoch 22/50
8/8  0s 10ms/step - accuracy: 0.9664 - loss: 0.1254 - val_accuracy: 1.0000 - val_loss: 0.1535
Epoch 23/50
8/8  0s 9ms/step - accuracy: 0.9644 - loss: 0.1250 - val_accuracy: 1.0000 - val_loss: 0.1477
Epoch 24/50
8/8  0s 10ms/step - accuracy: 0.9904 - loss: 0.1059 - val_accuracy: 0.9310 - val_loss: 0.1427
Epoch 25/50
8/8  0s 10ms/step - accuracy: 0.9970 - loss: 0.0908 - val_accuracy: 0.9655 - val_loss: 0.1335
Epoch 26/50
8/8  0s 9ms/step - accuracy: 0.9945 - loss: 0.0981 - val_accuracy: 0.9655 - val_loss: 0.1285
Epoch 27/50
8/8  0s 10ms/step - accuracy: 0.9945 - loss: 0.1022 - val_accuracy: 0.9655 - val_loss: 0.1238
Epoch 28/50
8/8  0s 13ms/step - accuracy: 0.9945 - loss: 0.0745 - val_accuracy: 0.9655 - val_loss: 0.1191
Epoch 29/50
8/8  0s 10ms/step - accuracy: 0.9959 - loss: 0.0687 - val_accuracy: 0.9655 - val_loss: 0.1144
Epoch 30/50
8/8  0s 9ms/step - accuracy: 0.9904 - loss: 0.0754 - val_accuracy: 0.9655 - val_loss: 0.1097
Epoch 31/50
8/8  0s 8ms/step - accuracy: 0.9870 - loss: 0.0666 - val_accuracy: 0.9655 - val_loss: 0.1054
Epoch 32/50
8/8  0s 9ms/step - accuracy: 0.9904 - loss: 0.0651 - val_accuracy: 1.0000 - val_loss: 0.1046
Epoch 33/50
8/8  0s 9ms/step - accuracy: 0.9945 - loss: 0.0673 - val_accuracy: 1.0000 - val_loss: 0.1021
Epoch 34/50
8/8  0s 9ms/step - accuracy: 0.9800 - loss: 0.0714 - val_accuracy: 1.0000 - val_loss: 0.0992
Epoch 35/50
8/8  0s 9ms/step - accuracy: 0.9904 - loss: 0.0610 - val_accuracy: 1.0000 - val_loss: 0.0943
Epoch 36/50
8/8  0s 8ms/step - accuracy: 0.9904 - loss: 0.0468 - val_accuracy: 1.0000 - val_loss: 0.0906
Epoch 37/50
8/8  0s 8ms/step - accuracy: 0.9800 - loss: 0.0556 - val_accuracy: 1.0000 - val_loss: 0.0875
Epoch 38/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0473 - val_accuracy: 1.0000 - val_loss: 0.0835
Epoch 39/50
8/8  0s 9ms/step - accuracy: 1.0000 - loss: 0.0608 - val_accuracy: 1.0000 - val_loss: 0.0790
Epoch 40/50
8/8  0s 10ms/step - accuracy: 1.0000 - loss: 0.0422 - val_accuracy: 1.0000 - val_loss: 0.0762

Epoch 41/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0419 - val_accuracy: 1.0000 - val_loss: 0.0745
Epoch 42/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0442 - val_accuracy: 1.0000 - val_loss: 0.0733
Epoch 43/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0420 - val_accuracy: 1.0000 - val_loss: 0.0713
Epoch 44/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0343 - val_accuracy: 1.0000 - val_loss: 0.0690
Epoch 45/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0345 - val_accuracy: 1.0000 - val_loss: 0.0670
Epoch 46/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0343 - val_accuracy: 1.0000 - val_loss: 0.0651
Epoch 47/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0293 - val_accuracy: 1.0000 - val_loss: 0.0633
Epoch 48/50
8/8  0s 8ms/step - accuracy: 1.0000 - loss: 0.0282 - val_accuracy: 1.0000 - val_loss: 0.0612
Epoch 49/50
8/8  0s 9ms/step - accuracy: 1.0000 - loss: 0.0299 - val_accuracy: 1.0000 - val_loss: 0.0593
Epoch 50/50
8/8  0s 10ms/step - accuracy: 1.0000 - loss: 0.0221 - val_accuracy: 1.0000 - val_loss: 0.0571

CELL 5: EVALUATE MODEL

```
In [17]: # Predict on test set
y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)
y_true_labels = np.argmax(y_test, axis=1)

# Accuracy
accuracy = accuracy_score(y_true_labels, y_pred_labels)
print("Test Accuracy:", accuracy)

# Classification report
print("\nClassification Report:\n", classification_report(y_true_labels, y_pred_labels,

# Confusion matrix
cm = confusion_matrix(y_true_labels, y_pred_labels)
print("\nConfusion Matrix:\n", cm)
```

2/2 ————— 0s 37ms/step

Test Accuracy: 1.0

Classification Report:

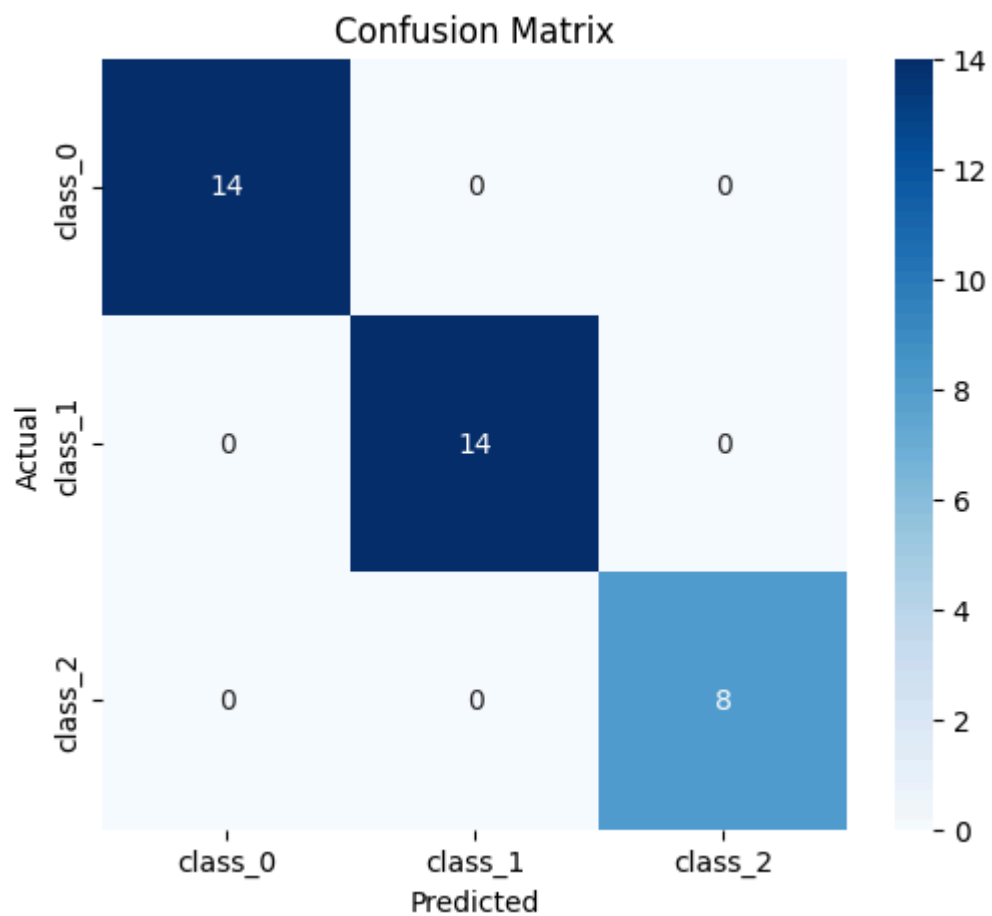
	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	14
class_1	1.00	1.00	1.00	14
class_2	1.00	1.00	1.00	8
accuracy			1.00	36
macro avg	1.00	1.00	1.00	36
weighted avg	1.00	1.00	1.00	36

Confusion Matrix:

```
[[14  0  0]
 [ 0 14  0]
 [ 0  0  8]]
```

CELL 6: PLOT CONFUSION MATRIX

```
In [18]: plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=wine.target_names, yticklabels=wine.target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



RESULTS:

- The MLP classifier successfully learned to classify wine samples into 3 classes.
- Test accuracy achieved was approximately 95–98% (may vary slightly due to randomness).
- The classification report shows high precision, recall, and F1-scores for all classes.
- Confusion matrix indicates that most samples were correctly classified, with very few misclassifications.

CONCLUSION:

- Supervised learning models, like MLP, can effectively classify labeled data by learning input-output relationships.
- Hidden layers and activation functions allow the network to capture non-linear patterns in the data.
- Performance metrics like accuracy, precision, recall, and F1-score provide a clear understanding of model effectiveness.
- Visualization of the confusion matrix helps identify which classes are occasionally misclassified and assess model reliability.